

CRYPTO MARKET MONITOR

Rapport Technique

Systeme de surveillance des marches crypto en temps reel

Auteurs	Adam Beloucif, Emilien Morice
Formation	M1 Data Engineering & IA - EFREI Paris
Module	Real-Time Engineering (S9 - 2025-2026)
Date	Juin 2026
Note	Projet realise a 2 personnes (consigne prevue pour 4)

1. Introduction

Ce rapport presente l'architecture, les choix techniques et les resultats du projet Crypto Market Monitor, realise dans le cadre du module Real-Time Engineering du M1 Data Engineering et IA de l'EFREI Paris (2025-2026).

L'objectif est de construire un pipeline de streaming bout-en-bout qui ingere des transactions crypto en direct depuis les flux WebSocket de Binance et Coinbase, les traite via Apache Kafka, calcule des metriques analytiques en fenetre glissante (VWAP, SMA-20, detection d'anomalies par z-score) et pousse les resultats vers un dashboard live via Socket.IO.

Le projet a ete realise en binome alors que la consigne prevoyait une equipe de quatre personnes.

2. Architecture du systeme

Le systeme suit un pattern pipeline lineaire : collecte -> streaming -> traitement -> diffusion. Chaque etape est isolee dans un service independant, communiquant exclusivement via Kafka ou Socket.IO.

2.1 Composants

Service	Role	Technologies
Ingester	Clients WebSocket Binance + Coinbase, normalisation, production	Node.js, TypeScript, ws, kafkajs
Kafka	Tampon de decoupling, tolerance aux pannes, multi-consommateurs	Apache Kafka 3.7 KRaft
Processor	Consomme les transactions, calcule VWAP SMA z-score	Node.js, TypeScript, kafkajs
API	Agregation, endpoints REST, push WebSocket clients	Express, Socket.IO, helmet
Dashboard	Graphiques prix/volume, flux anomalies, i18n FR/EN	HTML, CSS, JS, Chart.js 4

2.2 Flux de donnees

Sources WebSocket -> Ingester (normalisation) -> Kafka topic crypto-trades -> Processor (VWAP, SMA-20, z-score) -> Kafka topic crypto-metrics -> API Server -> Socket.IO -> Dashboard (push temps reel).

Le decoupling par Kafka permet aux producteurs et consommateurs de fonctionner a des debits differents sans perte de donnees. La retention est configuree a 1 heure par default. Le partitionnement se fait par symbole (cle = symbole), garantissant l'ordre des messages par paire de trading.

3. Apache Kafka - Mode KRaft

Le projet utilise Kafka 3.7 en mode KRaft, qui elimine la dependance a Zookeeper. Les metadonnees du cluster sont gereses en interne via le protocole Raft, ce qui simplifie le deploiement (1 conteneur + 1 volume) et reduit la surface d'attaque.

3.1 Image Docker

```
bitnami/kafka:3.7 (KRaft pre-configure, variable  
KAFKA_CFG_PROCESS_ROLES=broker,controller)
```

3.2 Topics

Topic	Description	Partitions	Retention
crypto-trades	Transactions brutes normalisees (interface Trade)	3	1h
crypto-metrics	Metriques calculees par symbole (VWAP, SMA, etc.)	3	1h

4. Services Node.js TypeScript

4.1 Ingester

L'ingester maintient des connexions WebSocket persistantes vers Binance et Coinbase. Chaque message entrant est deserialise, normalise vers l'interface Trade, puis publie sur le topic crypto-trades avec comme cle de partition le symbole.

- Binance : stream combine BTC/USDT + ETH/USDT via `wss://stream.binance.com:9443/stream`
- Coinbase : BTC-USD via `wss://advanced-trade-ws.coinbase.com`
- Reconnexion auto : backoff exponentiel, max 10 tentatives, delai max 30 secondes
- Arret propre SIGTERM/SIGINT avec vidage du buffer Kafka

4.2 Processor

Le processor consomme le topic crypto-trades et maintient un buffer circulaire en memoire (max 1000 trades/symbole, eviction TTL 10 minutes) par symbole. A chaque nouveau trade, il recalcule les metriques de la fenetre courante et publie le resultat sur crypto-metrics.

Metrique	Formule	Fenetre	Objectif
VWAP	$\text{sum}(\text{prix} \times \text{qte}) / \text{sum}(\text{qte})$	1 minute	Prix moyen pondere par volume
SMA-20	moyenne des 20 derniers prix	20 trades	Lissage de la tendance
Z-Score	$(\text{valeur} - \mu) / \sigma$	1 minute	Detection d'anomalie (seuil 2.5)
Var. 1min	$(\text{prix} - \text{prix}_0) / \text{prix}_0$	1 minute	Performance instantanee
Var. 5min	$(\text{prix} - \text{prix}_0) / \text{prix}_0$	5 minutes	Performance moyen terme
High/Low	max / min des prix	1 minute	Amplitude de la bougie

4.3 API Server

L'API Server consomme le topic `crypto-metrics`, maintient l'état courant en mémoire (dernières métriques par symbole + historique 200 points + 50 dernières anomalies) et expose deux interfaces :

- REST : 5 endpoints GET documentés (`health`, `metrics`, `metrics/:symbol`, `history/:symbol`, `anomalies`)
- WebSocket : Socket.IO v4.7 avec 5 types d'événements push (`metrics:update`, `anomaly:detected`, `server:stats`, `initial:state`, `subscribe:symbol`)
- Sécurité : `helmet`, `express-rate-limit` (100 req/15min/IP), CORS strict, morgan logging

5. Dashboard

Le dashboard est une application vanilla HTML/CSS/JS (sans framework) connectée au backend via Socket.IO. Les métriques sont mises à jour en temps réel par push serveur, sans polling.

5.1 Composants visuels

- Section hero : prix actuel avec flash vert/rouge au changement, badges variation 1min/5min
- Stat-cards : volume (1min), nombre de transactions, high/low de la bougie courante
- Graphique prix : ligne prix + SMA-20 superposée (Chart.js 4.4, 200 points max)
- Graphique volume : barres par paire (BTC-USDT, ETH-USDT, BTC-USD)
- Flux anomalies : liste déroulante avec icône alerte, valeur z-score, beep AudioContext
- Header : horloge temps réel, statut de connexion (live/demo), bascule FR/EN

5.2 Design system

Le design reprend l'identité visuelle de l'EFREI Paris : fond navy (#0A0E1A), accent amber/or (#F59E0B), typo Inter. Une barre de gradient amber-violet en haut de page sert de signature institutionnelle. Les logos crypto (Bitcoin, Ethereum, Coinbase) sont des SVG officiels.

5.3 Mode démonstration

Si le backend est inaccessible après 3 secondes (pas de connexion Socket.IO établie), le mode demo s'active automatiquement. Des données simulées réalistes sont injectées (prix de base BTC ~67k USD, ETH ~3.5k USD, variations aléatoires de +/-0.15%), avec mise à jour toutes les 800ms. Un badge 'DEMO' ambra s'affiche dans le header.

6. Internationalisation (i18n)

Le dashboard supporte le français (langue par défaut) et l'anglais. L'implémentation est un module vanilla JS (IIFE i18n) sans dépendance externe.

Principe de fonctionnement

- Chargement asynchrone du fichier i18n/{locale}.json au démarrage
- Tous les éléments statiques portent un attribut data-i18n (cle de traduction)
- La langue est persistée dans localStorage sous la cle cmm-locale
- document.documentElement.lang est mis à jour pour les technologies d'assistance
- Les fonctions formatPrice() et formatTime() utilisent l'API Intl pour le formatage locale-aware

7. Sécurité

helmet

Configuration des HTTP response headers : Content-Security-Policy, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy.

express-rate-limit

Limite de 100 requêtes par fenêtre de 15 minutes par adresse IP. Protection contre le brute-force et les attaques DDoS applicatives.

zod

Validation stricte des variables d'environnement au démarrage du service. Le processus s'arrête si une variable obligatoire est absente ou invalide.

CORS strict

Whitelist explicite des origines autorisées. Pas de wildcard (*) en production.

morgan

Logging HTTP avec rotation de fichiers. Aucune stack trace n'est exposée dans les réponses HTTP côté client.

Docker non-root

Tous les conteneurs tournent avec un utilisateur applicatif (UID 1000, appuser). Le dossier applicatif appartient à cet utilisateur.

.dockerignore

Les fichiers .env, node_modules, secrets et fichiers de développement sont exclus des images Docker.

Secrets

Zero secret en dur dans le code source. Toutes les variables sensibles sont injectées via des variables d'environnement. Le .env est gitignore.

8. Options de deployment

8.1 Docker Compose (recommande pour le developpement)

6 services independants avec healthchecks et restart on-failure : kafka, kafka-ui, ingester, processor, api, dashboard (nginx). Le dashboard est accessible sur le port 8080, Kafka UI sur 8090, l'API sur 3001.

```
docker-compose up --build -d
```

8.2 Image tout-en-un (demonstration / production)

Une image Docker unique (all-in-one.Dockerfile) embarque Apache Kafka (Java 17), les 3 services Node.js et nginx, geres par supervisord. Cela permet un deployment en une seule commande sans Docker Compose.

```
docker build -f all-in-one.Dockerfile -t crypto-monitor .
```

```
docker run -p 8080:8080 -p 3001:3001 crypto-monitor
```

8.3 Lanceur autonome (.exe)

Un script Node.js ESM (launcher/src/index.mjs) orchestre le demarrage : verification de Docker, build des images, attente des healthchecks, ouverture du navigateur, streaming des logs. Il peut etre compile en executable standalone via pkg :

```
cd launcher && npm install && npm run build:win # Windows .exe
```

```
cd launcher && npm install && npm run build:linux # Linux binaire
```

8.4 Variables d'environnement cles

Variable	Valeur par defaut	Description
KAFKA_BROKERS	localhost:9092	Serveurs bootstrap Kafka
KAFKA_TOPIC_TRADES	crypto-trades	Topic des transactions brutes
KAFKA_TOPIC_METRICS	crypto-metrics	Topic des metriques traitees
API_PORT	3001	Port du serveur API
SMA_WINDOW	20	Taille de la fenetre SMA
ANOMALY_ZSCORE_THRESHOLD	2.5	Seuil z-score (anomalie)
HISTORY_WINDOW_SECONDS	300	Retention historique (5 min)

9. Bilan et perspectives

Ce qui a été réalisé

- Pipeline Kafka bout-en-bout fonctionnel (Ingestion WS -> Kafka -> processing -> dashboard)
- 3 services TypeScript avec typage strict et tests unitaires de base
- Dashboard dark theme EFREI avec Chart.js, i18n FR/EN et mode demo offline
- Sécurité niveau production : helmet, rate-limiting, zod, CORS strict, non-root Docker
- 3 options de déploiement : Docker Compose, image tout-en-un, lanceur .exe
- Projet réalisé à 2 personnes pour une consigne prévue à 4

Perspectives d'évolution

- Ajout de paires supplémentaires : SOL/USDT, BNB/USDT, XRP/USDT
- Alertes email ou SMS via webhook (Kafka Connect + plugin SMTP/Twilio)
- Stockage historique dans InfluxDB ou TimescaleDB pour le backtesting
- Déploiement Kubernetes avec Helm chart pour la scalabilité horizontale
- Authentification JWT pour les endpoints REST et les connexions Socket.IO
- Tests d'intégration complets avec Testcontainers (Kafka réel en CI)

Crypto Market Monitor - Rapport Technique

Adam Beloucif, Emilien Morice - M1 Data Engineering & IA - EFREI Paris - 2025-2026